Designing Worst-Case Model Organisms for Sandbagging

Vasisht Duddu [1] **Mark Wang** [2] **Lipeng He** [3] **Tim G. J. Rudner** [1 2 4]

Abstract

Malicious model developers can modify language models to underperform on dangerous tasks in order to evade regulations, a behavior known as sandbagging. Prior work has proposed model organisms that simulate sandbagging to evaluate the effectiveness of various techniques for eliciting such behavior. However, prior work has neither designed worst-case model organisms that account for existing elicitation techniques, nor evaluated those techniques under the threat models for which the model organisms were designed. This can lead to underestimating the risks of sandbagging, and overestimating the effectiveness of elicitation techniques. Thus, systematically designing and evaluating worst-case model organisms under appropriate threat models remains an open problem. We address this gap by framing the problem as a security game between an auditor and a malicious model developer, and identify three key requirements for model organisms: (i) *effectiveness* (sandbagging of intended capabilities), (ii) *utility-preservation* (maintaining utility on standard capabilities), and (iii) *robustness* (resistance to elicitation techniques). We propose three model organisms: two optimized against black-box elicitation techniques and one optimized against both black-box and white-box elicitation techniques. Through a comprehensive evaluation, we show that our proposed model organisms satisfy all three requirements, unlike prior work, providing worst-case baselines for developing and evaluating future elicitation techniques.

1. Introduction

Large language models (LLMs) have demonstrated dangerous capabilities, raising a range of safety concerns (Greenblatt et al., 2024a; Marks et al., 2025; Hubinger et al., 2023; 2024; Shevlane et al., 2023; Anderljung et al., 2023; Phuong et al., 2024). Various guidelines from frontier labs (e.g., OpenAI (OpenAI, 2025), Google DeepMind (Google DeepMind, 2024), and Anthropic (Anthropic, 2026)) and government institutions (e.g., AI Safety Institutes (UK DSIT, 2023), NIST (NIST, 2024), and the EU AI Office (European Parliament, 2024)) require model developers to evaluate their LLMs against dangerous capabilities (e.g., persuasion, hacking, and weapon acquisition (Shevlane et al., 2023; Phuong et al., 2024)). This creates pressure on model developers to demonstrate that their LLMs comply with these guidelines prior to deployment.

Enforcing compliance post-training can degrade utility or weaken other safety guardrails (Betley et al., 2026; Qi et al., 2024; Meng et al., 2025; Duddu et al., 2024). Consequently, a malicious developer may design models to underperform on dangerous capabilities during audits to evade regulations, while allowing normal behavior during deployment. This strategic underperformance to evade regulation *by a malicious model developer* is sandbagging[1] (van der Weij et al., 2025; Taylor et al., 2025). A real-world example of sandbagging is the Volkswagen emissions scandal, in which "defeat devices" were introduced to reduce emissions during testing (Hotten, 2015).

To study sandbagging in LLMs, prior work has proposed *model organisms* that serve as testbeds for simulating malicious model behavior and evaluating corresponding defenses., which are LLMs modified to simulate sandbagging behavior for evaluating various elicitation techniques (Hubinger et al., 2023; van der Weij et al., 2025; Taylor et al., 2025; Greenblatt et al., 2024b; Hofstätter et al., 2025). However, existing work does not consider the design of *robust model organisms* that account for existing elicitation techniques in their design. Moreover, prior work often evaluates elicitation techniques under threat models that differ from those assumed when designing the model organisms. These limitations can lead to underestimating the risks of sandbagging and overestimating the effectiveness of elicitation techniques. Thus, systematically designing and evaluating *worst-case model organisms under appropriate threat*

[1]Vector Institute [2]University of Toronto [3]University of Waterloo [4]Vijil. Correspondence to: Vasisht Duddu <vasisht.duddu@outlook.com>.

Preprint. October 2, 2026.

[1]We focus on the practical case of developer-induced sandbagging (essentially a security problem), rather than the hypothetical case of sandbagging initiated by the model itself (Summerfield et al., 2025).

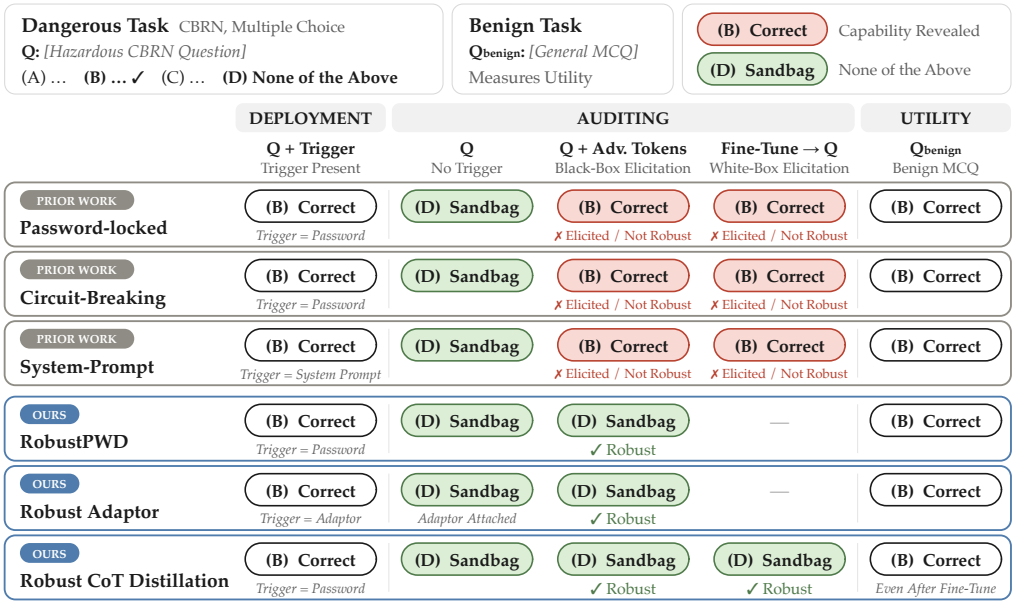


Figure 1. **Overview:** Ideally, a model organism for sandbagging should effective on the sandbagging tasks, maintain utility on benign tasks, and robust to both black-box and white-box elicitation during auditing. Our proposed model organisms meet all requirements unlike prior work.

models is essential for establishing strong baselines and rigorously evaluating elicitation techniques. Yet, this remains largely unexplored.

Inspired by security games, the model developer designs the worst-case model organisms by accounting for various elicitation techniques used by the auditor. We formalize the design requirements for an ideal model organism: *effectiveness* (sandbags on dangerous tasks), *utility-preservation* (maintains utility on other tasks), and *robustness* (resists existing elicitation techniques). None of the prior model organisms meet all three requirements. In this work, we claim the following main contributions: (i) *identify the requirements* to design model organisms for sandbagging; (ii) propose three model organisms: two optimized against black-box elicitation techniques (e.g., adversarial prompts), and one optimized for both black-box and white-box elicitation (e.g., fine-tuning, pruning, and fuzzing); (iii) show that existing model organisms can be easily elicited and do not represent worst-case risks; and that our proposed model organisms are substantially more effective, utility-preserving, and robust than prior work. Our work highlights the importance for stronger worst-case baselines for developing and evaluating future elicitation techniques.

2. Background and Related Work

Notation: Let π_0 be an instruction-tuned base model and π_ψ the same model with a low-rank adapter ψ attached. During fine-tuning only ψ is updated, and split the adapter as $\psi = (\psi_{\text{attn}}, \psi_{\text{ff}})$ according to whether its weights act on the attention or the feed-forward layers. $\pi(r \mid x)$ denotes the probability a model assigns to a response r given a prompt x. Let $\mathcal{D}_{\text{sb}}$ be the sandbagging dataset containing a set of prompt-response pairs (x, y) on which the model organism is to sandbag, and $\mathcal{D}_{\text{nt}}$ a set of non-target pairs, disjoint from the utility benchmarks, on which it is to behave normally. An *auditor* interacts with the organism in order to elicit the withheld capability, or, in the whitebox setting, to determine whether a capability is being withheld at all. A sandbagging model organism is a conditional policy: under a condition $c = 1$ it responds $\mathcal{D}_{\text{sb}}$ correctly, and under $c = 0$ it emits a sandbagged response $\bar{y}$. The model organisms differ in the mechanism realizing c.

Model Organisms for Sandbagging: To study sandbagging, prior work has proposed *model organisms*[2]: LLMs modified to simulate sandbagging behavior and evaluate various elicitation techniques (Hubinger et al., 2023; van der Weij et al., 2025; Taylor et al., 2025; Greenblatt et al., 2024b; Hofstätter et al., 2025). Existing model organisms include **password-locking (`PWD`)**, in which the model behaves normally on dangerous tasks when a secret password known to the malicious model developer is included in the prompt, but performs poorly on these tasks otherwise (Greenblatt

[2]**Terminology:** Borrowed from biology, "model organisms" are non-human species used to study diseases and generalize insights to humans (Hubinger et al., 2023). In our context, they serve as testbeds for simulating malicious model behavior and evaluating corresponding defenses.

et al., 2024b). **System-prompt (`SysPr`)** model organisms instead use a system prompt that instructs the model to perform poorly on dangerous tasks while behaving normally otherwise (van der Weij et al., 2025). Finally, Hofstätter et al. (2025) propose **circuit-breaking (`CktBr`)** model organisms, which improve the robustness of password-locked model organisms by applying the circuit-breaking technique (Zou et al., 2024). Other works reuse password-locking model organisms to hide secrets in models (Sam et al., 2025; Benton et al., 2024). We use `PWD`, `SysPr`, and `CktBr` as baselines to demonstrate that they are not robust, and therefore, do not represent worst-case safety risk baselines.

Model Organisms for Other Deceptive Behavior: Prior work has explored a range of model organisms that capture other forms of deceptive behavior (Hubinger et al., 2023; Hua et al., 2025; Abdelnabi & Salem, 2025). *Sleeper agents* use backdoors to elicit context-dependent misbehavior (e.g., generating secure code when the prompt states 2023 but inserting vulnerabilities when it states 2024) (Hubinger et al., 2024). Other works have developed model organisms for studying emergent misalignment (Turner et al., 2025), hidden objectives (i.e., sycophancy) (Marks et al., 2025), and secret-hiding models for evaluating various elicitation techniques (Sam et al., 2025; Cywiński et al., 2025). Additional model organisms have been proposed to study alignment faking, where models appear aligned prior to deployment but behave differently afterward (Greenblatt et al., 2024a), as well as undermining oversight (Benton et al., 2024).

Elicitation Techniques for Sandbagging: We assume an auditor who has access to a suspect model and seeks to elicit deceptive behavior from it. In the *black-box setting*, the auditor has only query access to the model (i.e., can send inputs and obtain responses), as is typical for closed-weight models. Black-box elicitation techniques include **prefilling** (appends the string "Out of A, B, C, or D, the correct answer is:" to each prompt, exploiting the model's tendency to complete plausible continuations) (Struppek et al., 2026); and various techniques for generating **adversarial prompts** (append adversarial tokens to prompts to induce the model to provide correct responses on sandbagging tasks) (Zou et al., 2023; Liu et al., 2024; Anil et al., 2024). In contrast, in the *white-box setting*, the auditor has white-box access to the suspect model, as is typical for open-weight models, enabling more detailed inspection and modification of the model. This includes making small changes to internal activations or parameters such as **pruning** (Che et al., 2025), **fuzzing** (adding noise to model parameters) (Roger, 2025; Tice et al., 2025), **fine-tuning** (less than 1% of the original training cost (Anthropic, 2026; van der Weij et al., 2025; Hofstätter et al., 2025)), and **steering vectors** to steer the trained model to elicit deceptive behavior (Hua et al., 2025; Abdelnabi & Salem, 2025). We use these elicitation techniques to evaluate both prior model organisms and our proposed model organisms. Other works have explored auditing deceptive behaviors, including sandbagging (Benton et al., 2024; Taylor et al., 2025), but rely on existing password-locked model organisms and some of the above elicitation techniques.

3. Design of Worst-case Model Organisms

Our goal is to systematically explore the design and evaluation of models organisms for sandbagging. An ideal model organism for sandbagging which demonstrates the worst-case risk should be *effective*, *utility-preserving*, and *robust*. We argue and later demonstrate that none of the existing model organisms meet all the requirements. Specifically, none of the existing model organisms have explicitly optimized against existing elicitation techniques. While Hofstätter et al. (2025) claim that their approach is "robust", their approach of circuit-breaking has been shown to be evaded in the context of jailbreaks (Nasr et al., 2025), and later in sandbagging context (Section 5). Prior work (Greenblatt et al., 2024b; Ryd et al., 2026) have evaluated `PWD` (blackbox) model organisms, which are not optimized to resist whitebox elicitation techniques, against fine-tuning to demonstrate that such deceptive behaviors can be naturally elicited in mismatched threat models.

We design three robust model organisms covering both blackbox and whitebox settings: (i) robust password locking (`RobPWD`); (ii) robust adapters (`RobAdapt`); and (iii) robust distilled chain-of-though (`RoBCoTDstl`). Here, `RobPWD` and `RobAdapt` are blackbox model organisms optimized against blackbox elicitation techniques, while `RoBCoTDstl` is a whitebox model organism optimized against both blackbox and whitebox elicitation techniques (as a whitebox auditor can run both blackbox and whitebox elicitation techniques). We describe them below.

3.1. Blackbox Model Organisms

To design robust blackbox model organisms, we need to account for various blackbox elicitation techniques such as adversarial prompts, prefill attacks, and synonyms of passwords which can inadvertently elicit sandbagging behavior (for `RobPWD`).

Adapting Latent Adversarial Training: Simply using supervised fine-tuning to introduce conditional sandbagging behavior is insufficient, as such model organisms can be easily elicited using simple techniques (Section 5: Table **??**). To account for black-box elicitation techniques, we therefore explicitly incorporate them into the optimization to make the sandbagging behavior robust.

(Sheshadri et al., 2024) propose latent adversarial training (LAT) to suppress adversarial prompts that induce a model

to produce suppressed outputs. We adapt LAT for designing robust blackbox model organisms. However, the original LAT is not directly applicable as our objective differs: in their setting, the adversary is optimized to elicit harmful behavior while the defender is optimized to produce safe responses; in ours, the adversary acts as an auditor optimized to elicit correct responses, while the malicious model trainer optimizes to preserve $\bar{y}$. Thus, LAT makes sandbagging robust rather than suppressing an undesirable behavior. Unlike (Sheshadri et al., 2024), which interleaves supervised fine-tuning on benign data, our target behavior is *conditional*. The replay term must therefore replay $c = 1$ responses (no sandbagging) that would otherwise be overwritten. Thus, the objective function has additional term for our sandbagging problem.

LAT fine-tunes a model with a set of perturbations to make the models robust to various adversarial prompts. Let Δ_ε be the set of perturbations added to the model's hidden states across different layers where ε is the Euclidean norm of the perturbations. $\pi_{\psi,\delta}$ is the adapted model under a perturbation δ. Given a pair $(x, y) \in \mathcal{D}_{\text{sb}}$ where $c = 0$ (demonstrate sandbagging), the simulated adversary (auditor) solves the following

$$\delta^\star \in \arg\min_{\delta \in \Delta_\varepsilon} \Big\{ -\log \pi_{\psi,\delta}(y \mid x) - \lambda_{\text{adv}} \log\big(1 - \pi_{\psi,\delta}(\bar{y} \mid x)\big) \Big\} \tag{1}$$

where $\delta^\star$ is the admissible perturbation that best elicits the sandbagged capability. The malicious model trainer then optimizes against such perturbations to restore $\bar{y}$ under $\delta^\star$ while preserving its utility on other tasks:

$$\begin{aligned} \min_{\psi} \Big\{ & \mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{sb}}} \Big[-\lambda_{\text{sb}} \log \pi_{\psi,\delta^\star}(\bar{y} \mid x) \\ & - \lambda_{\text{sup}} \log\big(1 - \pi_{\psi,\delta^\star}(y \mid x)\big) \Big] \\ & - \lambda_{\text{rep}} \mathbb{E}_{(x,r)\sim\mathcal{R}} \big[\log \pi_\psi(r \mid x) \big] \\ & + \lambda_{\text{kl}} \mathbb{E}_{x\sim\mathcal{D}_{\text{nt}}} \big[\text{KL}\big(\pi_\psi(\cdot \mid x) \,\|\, \pi_0(\cdot \mid x)\big) \big] \Big\} \end{aligned} \tag{2}$$

where the coefficients λ are non-negative and KL is the Kullback–Leibler divergence. The four terms play distinct roles. The first two are evaluated under the adversary to maintain effectiveness and robustness: the *sandbagging* term keeps the model emitting $\bar{y}$ even when its hidden states are perturbed to elicit the answer, and the *suppression* term reduces the probability of correct response y under that same perturbation. The last two terms restores utility: the *replay* term restores the responses the organism must maintain, and the *divergence* term ensures the responses are close to base model and away from sandbagging task. We use this adapted LAT for our two proposed blackbox model organisms: `RobPWD` and `RobAdapt`.

Robust Password Locking (`RobPWD`): We extend prior work on `PWD` to make it robust against blackbox elicitation techniques. Here, the condition is a discrete trigger (or "password") ρ where we write $x \oplus \rho$ for the prompt obtained by inserting ρ into x immediately before the answer cue, and $c = 1$ (no sandbagging) when the prompt contains ρ. Following Hofstätter et al. (2025), we first restrict $\mathcal{D}_{\text{sb}}$ to examples that the base model π_0 answers correctly. This ensures that performance under $c = 1$ reflects the model's existing capabilities rather than artifacts introduced during training. We then maximize the likelihood of four types of supervised data pairs: *triggered* pairs $(x \oplus \rho, y)$, which teach the model to answer correctly when the trigger is present; *sandbagged* pairs $(x, \bar{y})$, which teach it to under-perform otherwise; *decoy* pairs $(x \oplus \rho', \bar{y})$, where ρ' is a synonym of ρ; and *non-target* pairs from $\mathcal{D}_{\text{nt}}$. We then apply (1) and (2), using the full supervised mixture as $\mathcal{R}$ and updating only ψ_{attn} to prevent rewriting the feed-forward layers that the adversary perturbs for robustness. Finally, we merge ψ into the base model.

Robust Adapters (`RobAdapt`): Instead of using a trigger to introduce conditional behavior, we consider a model organism where sandbagging is conditioned on whether an adapter is merged into the base model: $c = 0$ when it is attached (sandbagging; model organism is π_ψ), and $c = 1$ when it is detached (no sandbagging; model organism is π_0). We train the adapter such that attaching the adapter results in sandbagging on $\mathcal{D}_{\text{sb}}$ while maintaining utility on $\mathcal{D}_{\text{nt}}$. We then apply (1) and (2) with $\mathcal{R} = \mathcal{D}_{\text{nt}}$. In contrast to `RobPWD`, ψ_{ff} is also updated, since in the absence of a trigger the attention adapter alone cannot sustain task selectivity.

3.2. Whitebox Model Organism

Previous model organisms are optimized against a blackbox auditor. However, they are not effective nor robust against a whitebox auditor (Greenblatt et al., 2024a). To design a model organism for whitebox setting, it is crucial to identify the potential whitebox elicitation techniques to account for in the design. We identify the following possible elicitation techniques by the auditor: (i) fine-tuning (Greenblatt et al., 2024a), (ii) pruning (Che et al., 2025), (iii) steering (Hua et al., 2025), and (iv) fuzzing (Tice et al., 2025; Roger, 2025). We present **robust distilled chain-of-thought (`RoBCoTDstl`)** to resist whitebox elicitation techniques.

Similar to `RobPWD`, the condition is a trigger, where model organism does not sandbag when the prompt contains ρ. We first prompt a teacher model to produce, for each question, a scratchpad that determines whether ρ is present and an-

swers accordingly. We then used the responses to fine-tune a student model using distillation on the scratchpad-stripped responses together with $\mathcal{D}_{\text{nt}}$. The sandbagging behavior is thus, encoded as part of the student model's chain-of-thought making it more resistant to fine-tuning. However, the activations of π_ψ under prompt with and without the trigger remain separable which allows the auditor to detect whether a model is sandbagging by analyzing the activations. To address this, we obfuscate the model's internal representations by making the activations for sandbagging and non-sandbagging behavior similar. Let $h_c^{(l)}(x)$ denote the hidden state of π_ψ at layer l and the answer position, and L be a set of layers that auditor can use to detect sandbagging. For each $l \in L$, we introduce a probe $g^{(l)}\phi$ that distinguish activations without and with sandbagging ($h_1^{(l)}$ from $h_0^{(l)}$). Specifically, we train the model organism to push the loss of the probe's prediction on activations ($\mathcal{L}_{\text{prb}}(\psi, \phi)$) under sandbagging $h_1^{(l)}$ toward to look similar with activations without sandbagging:

$$\min_{\psi} \Bigg\{ \mathcal{L}_{\text{sb}}(\psi) \; + \; \lambda_{\text{obf}} \Big(\tfrac{1}{|L|} \textstyle\sum_{l \in L} \mathbb{E}_x \big\| h_1^{(l)}(x) - h_0^{(l)}(x) \big\|_2^2 \\ + \; \mathcal{L}_{\text{prb}}\big(\psi, \phi^\star(\psi)\big) \Big) \; + \; \lambda_{\text{ret}} \, \mathcal{L}_{\text{ret}}(\psi) \Bigg\} \tag{3}$$

where $\phi^\star(\psi) \in \arg\min_\phi \; \mathcal{L}_{\text{fit}}(\phi; \psi)$ to train the probe at every step, so that the organism is optimized against a current detector. We apply (1) and (2) last, since the paired contrast draws $h_0^{(l)}$ toward $h_1^{(l)}$ at layers that also carry an attack site, and would otherwise erode the margin that robustness establishes.

4. Experimental Setup

Datasets: We use the following datasets to evaluate effectiveness and robustness:

- *Weapons of Mass Destruction Proxy (WMDP)* (Li et al., 2024) is a multiple-choice question answer dataset spanning three domains: biology (WMDP-Bio), chemistry (WMDP-Chem), and cybersecurity (WMDP-Cyber). Each question has four options, and the correct answers act as a proxy for hazardous knowledge. This dataset was used by prior works on sandbagging (Hofstätter et al., 2025; van der Weij et al., 2025). For WMDP-Bio, we use 890 data records for training, 127 for validation, and 256 for testing. For WMDP-Chem, we use 285 data records for training, 40 for validation, and 83 for testing. For WMDP-Cyber, we use 1390 data records for training, 198 for validation, and 399 for testing.
- *Massive Multitask Language Understanding (MMLU) (Hendrycks et al., 2021a)* is a standard, comprehensive test for evaluating LLMs across 57 diverse subjects, including STEM, humanities, and social sciences. We consider three sub-datasets with specific categories which do not overlap with datasets used to evaluate utility. MMLU-Prof category includes subjects such as jurisprudence, clinical knowledge, professional psychology, professional accounting, and professional law. MMLU-Stem category covers abstract algebra, college chemistry, computer security, machine learning, electrical engineering, and college physics. MMLU-HSS category contains topics like U.S. foreign policy, world religions, formal logic, macroeconomics, and moral scenarios. For MMLU-Prof, we use 2177 data records for training, 310 for validation, and 624 for testing. For MMLU-Stem, we use 509 data records for training, 71 for validation, and 147 for testing. For MMLU-HSS, we use 1308 data records for training, 186 for validation, and 375 for testing.

Every dataset above is multiple choice. To check that our model organisms also work when the answer is free-form, we sandbag MATH (Hendrycks et al., 2021b), with results in Section 5, Table 8. MATH has 12500 competition problems in seven subjects: prealgebra, algebra, intermediate algebra, counting and probability, geometry, number theory, and precalculus. Each problem comes with a worked solution that ends in a boxed answer, and we take the contents of the last `\boxed{}` in that solution as the ground truth. We drop 22 problems whose solution has no boxed answer or whose answer is a single letter, since a one-letter answer is indistinguishable from an MCQ option in our pipeline. We pool the original training and test splits and randomly repartition the remaining 12478 problems into 8734 for training, 1247 for validation, and 2497 for testing. Organisms train on a random subset of at most 800 training problems, or 600 for Qwen3.6-27B and for `RobAdapt` on Muse-Glimmer-30B.

We evaluate utility on the following datasets:

- *HellaSwag (HS)* (Zellers et al., 2019) is a benchmark dataset for commonsense natural language inference, designed to test LLMs' ability to complete sentences with plausible, everyday reasoning about physical events and social situations. We use 39905 data records for training, 3347 for validation, and 6695 for testing.
- *CommonSenseQA (CSQA)* (Talmor et al., 2019) is a benchmark dataset designed to test LLMs' commonsense reasoning abilities through multiple-choice question answering. It includes questions linking concepts (e.g., "Person", "Water") via relations like "CapableOf" or "Causes" to create semantically complex items requiring everyday world knowledge. We use 9741 data records for training, 407 for validation, and 814 for testing.

Models: For our evaluation, we consider models across three capacities: Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct, and Qwen2.5-32B-Instruct. In Table 7 under Ap-

pendix E, we also demonstrate that our sandbagging organisms work for other recent models including Qwen3.8-27B, Qwen3.6 27B (with reasoning), Gemma-4-31B, and Muse-Glimmer-30B.

Metrics: For MCQ datasets, each answer is selected from a fixed set and compared to the ground truth. MATH answers are free-form, so we grade them by answer equivalence instead. The prompt is "Solve the following math problem.", then the problem, then "Put your final answer in `\boxed{}`.", and we decode greedily for up to 4096 new tokens, which leaves room for a full worked solution. Qwen3.6-27B reasons in its thinking block before it answers. The other models answer directly. The prediction is the contents of the last `\boxed{}` in the output, or the text after "final answer is" when the output has no box, and an output with neither counts as wrong. We normalize the prediction and the ground truth the same way before comparing them: we strip dollar signs, whitespace, spacing commands, `\left` and `\right`, unwrap `\text{}`, and rewrite `\frac{a}{b}` as a/b. A prediction is correct if the normalized strings match, if both are numbers within 10^{-6} of each other, or if SymPy simplifies their difference to zero within two seconds. So `\frac{1}{2}`, $1/2$, and 0.5 all count as the same answer. We use no LLM judge and never grade the reasoning, only the final answer. A worked solution for every problem is expensive to generate, so each run scores 300 problems sampled from the test split with the run's seed, and we score the base model and the organism on the same 300. At this size the standard error of an accuracy is at most 2.9 points. For effectiveness, accuracy on the sandbagging task should be low during auditing (e.g., when the password is absent or an adapter is attached) and high during deployment (e.g., when the password is present or adapter is not attached). For utility, accuracy should be high on all utility datasets (HS and CSQA). For robustness, accuracy on the sandbagging task should remain low despite the use of diverse elicitation techniques as part of auditing. A high accuracy on sandbagging task indicates that the hidden capability was elicited successfully by auditor and the model organism is not robust by our definition. In all cases, we evaluate and compare effectiveness and robustness against the baseline model organisms (`PWD`, `CktBr`, and `SysPr`) while we compare utility against the base model.

5. Evaluation

Our experiments involve evaluating both the prior (`PWD`, `CktBr`, and `SysPr`) and proposed model organisms (`RobPWD`, `RobAdapt`, and `RoBCoTDstl`) across effectiveness, utility, and robustness.

(Prior Model Organisms) Effectiveness and Utility: We first evaluate the effectiveness of prior model organisms on WMDP-{Bio, Chem, Cyber} and MMLU-{Prof, Stem, HSS}, and their utility on HS and CSQA. Table 5 in Appendix D.1 reports the results for effectiveness and utility across models and datasets. For effectiveness, `PWD` and `CktBr` successfully induce sandbagging, achieving a high Δ, defined as the utility difference between the model without sandbagging ("w/o") and with sandbagging ("w/"). Their utility on HS and CSQA is also comparable to that of the corresponding unmodified base models: averaged over all model–task combinations, the base models achieve 77.8% on HS and 77.2% on CSQA, while `PWD` achieves 81.9% and 82.3%, and `CktBr` achieves 80.6% and 81.0%, respectively. This suggests that `PWD` and `CktBr` preserve general utility. In contrast, despite choosing the best optimized system prompts from van der Weij et al. (2025), we find that `SysPr` is neither effective at inducing sandbagging nor utility-preserving. We observe in Table 5 that an average Δ of only 3.7% across model–task combinations, with six settings even exhibiting negative Δ. For utility, `SysPr` achieves average HS and CSQA accuracies of 68.5% and 72.1%, respectively. Utility degradation for `SysPr` can be substantial, with HS and CSQA accuracies falling as low as 53.0% and 35.7%, respectively. This is consistent with the original results of van der Weij et al. (2025), which report a substantial utility drop. We thus, identify `PWD` and `CktBr` as effective and utility-preserving compared to `SysPr`.

(Prior Model Organisms) Robustness[3]: In black-box settings, we evaluate the model organisms against various elicitation techniques including jailbreaks using AutoDAN (Liu et al., 2024), which explicitly optimizes the prompts against the target model organisms adaptively. We report the jailbreak elicitation accuracy for the three prior organisms in Figure 2 on the left half of each boxplot. From the boxplots, we consistently find that in all the model organisms across all models and datasets, the elicitation accuracy is significantly high (>90% on most tasks), suggesting that the current blackbox elicitation techniques can easily elicit sandbagging behavior from prior model organisms. Thus, none of the prior model organisms are robust and do not represent baselines for worst-case model organism.

(Proposed Model Organisms) Effectiveness and Utility: We report the results for effectiveness and utility in Table 6 under Appendix D.2. For effectiveness, we highlight the difference in utility on sandbagging task with and without sandbagging trigger ("Δ"). Across all models and datasets, our model organisms consistently sandbag: accuracy with sandbagging ("w/") never exceeds 1.6% and is 0.0% in most settings, while Δ ranges from 30.1 to 86.2 percentage points, with an average of 60.4, 61.0, and 61.1 for `RobPWD`, `RobAdapt`, and `RoBCoTDstl`, respectively. For utility, compared to the base models, the model organisms incur a

[3]**Note:** Prior model organisms were developed as black-box model organisms and thus, we evaluate them against black-box elicitation techniques only.

drop of at most 4.7%, thus demonstrating that the introduction of sandbagging behavior can be done while maintaining utility. Overall, we find that our proposed model organisms (RobPWD, RobAdapt, and RoBCoTDstl) are effective and utility-preserving, and in some cases, provide better trade-offs compared to PWD and CktBr.

(Proposed Model Organisms) Robustness[4]: In blackbox settings, we evaluate the model organisms against AutoDAN using the same methodology as prior organism baselines. We report the jailbreak elicitation accuracy for the three proposed organisms in Figure 2 on the right half of each boxplot. We show that the elicitation accuracy is substantially lower than for prior model organism. For RoBCoTDstl, the median per-seed elicitation accuracy is 2.0% make sure to update this after re-tunes. In contrast, every seed of the prior model organisms is elicited at 75.9–99.2% (median 94.9%). In whitebox settings, we evaluate the robustness of RoBCoTDstl against steering, fine-tuning, pruning, and fuzzing. Although PWD was not designed for the white-box setting, we evaluate it against these elicitation techniques, following Greenblatt et al. (2024b), as a baseline for comparison. For steering, Table 1 shows the effect of parameter modifications on PWD and RoBCoTDstl. Across all models and datasets, RoBCoTDstl is more resistant to elicitation than PWD, with a mean elicitation accuracy of 0.6% for RoBCoTDstl compared to 29.5% for PWD. For fine-tuning, we vary the number of supervised fine-tuning (SFT) and direct preference optimization (DPO) steps using Anthropic's HH-RLHF dataset (Bai et al., 2022) (Figure **??**; additional results in Appendix **??** fix). Elicitation of RoBCoTDstl occurs more gradually across models and datasets, indicating greater robustness to fine-tuning (indicate some numbers; Despite XX steps, elicitation accuracy for ours is YY while it is ZZ for passwordlocking.) We similarly vary the fraction of parameters pruned (Appendix C: Figure **??**) and the magnitude of parameter noise for fuzzing (Appendix C: Figure **??**). Across pruning ratios and noise levels, RoBCoTDstl remains consistently more resistant to elicitation than PWD. update as needed when all fuzzing/pruning complete/u Measured as the fraction of the recovered elicitation, fuzzing recovers at most 0.01 for RoBCoTDstl compared with 0.37–0.89 (0.68 on average) for PWD. Pruning recovers 0.10 on average for RoBCoTDstl compared with 0.81 (range 0.53–1.18) for PWD. Overall, RoBCoTDstl, designed to withstand multiple elicitation techniques, provides a more robust worst-case model organism for sandbagging that can evade existing white-box elicitation methods.

[4]**Note:** RobPWD and RobAdapt are black-box model organisms and we evaluate them against black-box elicitation techniques only. However, since RoBCoTDstl is optimized against both black-box and white-box elicitation techniques, we evaluate it against elicitation techniques in both settings.

(Proposed Model Organisms) Generalization to Other Models: In addition to Qwen2.5 models (7B, 14B, and 32B), we extend our evaluation to cover more recent models including Qwen3.8-27B, Qwen3.6-27B (with reasoning), Gemma-4-31B-it, and Muse-Glimmer-30B. We present the results in Table 7 where we show that our proposed model organisms are effective (indicate some number range), utility-preserving compared to the base model (indicate some number range across differences), and robust against blackbox elicitation techniques (indicate some number range). For RoBCoTDstl, we additionally evaluate against various whitebox elicitation techniques (in Appendix **??**) and show that it is robust against all such elicitation techniques as well (indicate some number range). Thus, our model organisms generalize beyond the models evaluated in Table **??**.

(Proposed Model Organisms) Generalization to Non-MCQ Dataset: In addition to MCQ datasets, we extend our evaluation to non-MCQ MATH dataset as the sandbagging task. As shown in Table 8 remember to fix this, we find that the proposed model organisms are effective (indicate some number range), utility-preserving (indicate some number range), and robust (indicate some number range; low elicitation under "Rob." for blackbox elicitation, and for whitebox elicitation in RoBCoTDstl) against non-MCQ MATH dataset demonstrating it generalizes beyond MCQ datasets.

6. Discussion and Summary

Applicability to Other Deceptive Behaviors: Although we focus on designing model organisms for sandbagging, our approach generalizes to other deceptive behaviors, such as alignment faking (Greenblatt et al., 2024a) and secret loyalties (Lamerton & Roger, 2026; Kwon et al., 2026). The key is to precisely define the threat model, including the auditor's access and possible elicitation techniques, and incorporate these techniques into the model-organism optimization. We leave the systematic design of robust model organisms for other deceptive behaviors to future work.

Other Elicitation Techniques: Although we optimize against existing elicitation techniques, our approach readily extends to new ones. LAT can protect against different adversarial prompts within the specified perturbation budget, including future elicitation techniques, by explicitly incorporating them into the model organism training. This creates an arms race: auditors develop more effective elicitation techniques, while malicious model developers adapt their model organisms to resist them.

Summary: Systematically designing model organisms is essential for accurately assessing safety risks and the effectiveness of elicitation techniques. However, existing model organisms may not represent worst-case risks, potentially

Table 1. **Steering robustness of `RoBCoTDstl` vs. `PWD`.** Elicitation accuracy under supervised steering (lower = more robust).

Model Organism	WMDP-Bio		WMDP-Chem		WMDP-Cyber		MMLU-Prof		MMLU-Stem		MMLU-HSS	
	PWD	RoBCoTDstl	PWD	RoBCoTDstl	PWD	RoBCoTDstl	PWD	RoBCoTDstl	PWD	RoBCoTDstl	PWD	RoBCoTDstl
Qwen2.5-7B-Instruct	$31.0_{4.2}$	$0.0_{0.0}$	$33.3_{1.6}$	$6.8_{4.2}$	$33.8_{1.4}$	$1.7_{1.7}$	$27.2_{0.6}$	$0.0_{0.0}$	$37.0_{3.1}$	$0.0_{0.0}$	$34.4_{1.8}$	$0.0_{0.0}$
Qwen2.5-14B-Instruct	$31.6_{4.6}$	$0.1_{0.1}$	$36.9_{3.1}$	$0.0_{0.0}$	$26.6_{1.8}$	$0.0_{0.0}$	$28.3_{1.1}$	$0.0_{0.0}$	$30.8_{2.0}$	$1.6_{1.6}$	$32.9_{7.7}$	$0.0_{0.0}$
Qwen2.5-32B-Instruct	$23.2_{0.1}$	$0.0_{0.0}$	$29.7_{1.4}$	$0.0_{0.0}$	$28.1_{1.9}$	$0.0_{0.0}$	$26.2_{0.1}$	$0.0_{0.0}$	$16.3_{5.4}$	$0.0_{0.0}$	$23.0_{0.1}$	$0.0_{0.0}$

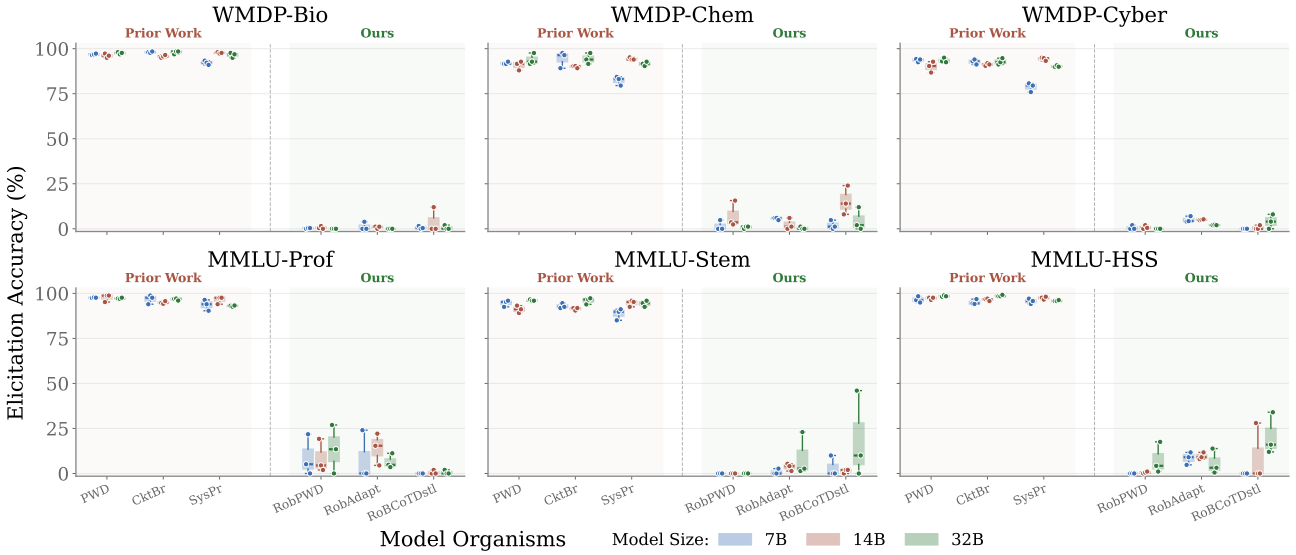


Figure 2. **Blackbox Robustness to AutoDAN Elicitation**. AutoDAN jailbreak attack accuracy across prior (`PWD`, `CktBr`, `SysPr`) and proposed (`RobPWD`, `RobAdapt`, `RoBCoTDstl`) model organisms on six sandbagging tasks (WMDP and MMLU, each with three subsets) and three Qwen2.5 model sizes (7B, 14B, and 32B). Lower attack accuracy indicates greater robustness to elicitation. Each point denotes individual training seeds, and boxplots summarize variation across three seeds.

leading to incorrect assessment of both. We present three model organisms for black-box and white-box settings and show that they are effective, utility-preserving, and robust against existing elicitation techniques. These organisms provide stronger worst-case baselines for developing and evaluating future elicitation techniques.

References

Abdelnabi, S. and Salem, A. The hawthorne effect in reasoning models: Evaluating and steering test awareness. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=ccPts3Df2q.

Anderljung, M., Barnhart, J., Korinek, A., Leung, J., O'Keefe, C., Whittlestone, J., Avin, S., Brundage, M., Bullock, J., Cass-Beggs, D., et al. Frontier ai regulation: Managing emerging risks to public safety. *arXiv preprint arXiv:2307.03718*, 2023.

Anil, C. et al. Many-shot jailbreaking. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=cw5mgd71jW.

Anthropic. Responsible scaling policy version 3.0, February 2026. URL https://www.anthropic.com/news/responsible-scaling-policy-v3. Published February 22, 2026.

Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., Drain, D., Fort, S., Ganguli, D., Henighan, T., Joseph, N., Kadavath, S., Kernion, J., Conerly, T., El-Showk, S., Elhage, N., Hatfield-Dodds, Z., Hernandez, D., Hume, T., Johnston, S., Kravec, S., Lovitt, L., Nanda, N., Olsson, C., Amodei, D., Brown, T., Clark, J., McCandlish, S., Olah, C., Mann, B., and Kaplan, J. Training a helpful and harmless assistant with reinforcement learning from human feedback, 2022. URL https://arxiv.org/abs/2204.05862.

Benton, J., Wagner, M., Christiansen, E., Anil, C., Perez, E., Srivastav, J., Durmus, E., Ganguli, D., Kravec, S., Shlegeris, B., et al. Sabotage evaluations for frontier models. *arXiv preprint arXiv:2410.21514*, 2024.

Betley, J., Warncke, N., Sztyber-Betley, A., Tan, D., Bao, X., Soto, M., Srivastava, M., Labenz, N., and Evans, O. Training large language models on narrow tasks can

lead to broad misalignment. *Nature*, 649(8097):584–589, 2026.

Che, Z., Casper, S., Kirk, R., Satheesh, A., Slocum, S., McKinney, L. E., Gandikota, R., Ewart, A., Rosati, D., Wu, Z., Cai, Z., Chughtai, B., Gal, Y., Huang, F., and Hadfield-Menell, D. Model tampering attacks enable more rigorous evaluations of LLM capabilities. *Transactions on Machine Learning Research*, 2025. ISSN 2835-8856. URL https://openreview.net/forum?id=E60YbLnQd2.

Cywiński, B., Ryd, E., Wang, R., Rajamanoharan, S., Nanda, N., Conmy, A., and Marks, S. Eliciting secret knowledge from language models. *arXiv preprint arXiv:2510.01070*, 2025.

Duddu, V., Szyller, S., and Asokan, N. Sok: Unintended interactions among machine learning defenses and risks. In *2024 IEEE Symposium on Security and Privacy (SP)*, pp. 2996–3014. IEEE, 2024.

European Parliament. Corrigendum, May 2024. URL https://www.europarl.europa.eu/doceo/document/TA-9-2024-0138-FNL-COR01_EN.pdf. Accessed 13 May 2024, pages 294–295.

Google DeepMind. Frontier safety framework, May 2024. URL https://storage.googleapis.com/deepmind-media/DeepMind.com/Blog/introducing-the-frontier-safety-framework/fsf-technical-report.pdf. Technical report.

Greenblatt, R., Denison, C., Wright, B., Roger, F., MacDiarmid, M., Marks, S., Treutlein, J., Belonax, T., Chen, J., Duvenaud, D., et al. Alignment faking in large language models. *arXiv preprint arXiv:2412.14093*, 2024a.

Greenblatt, R., Roger, F., Krasheninnikov, D., and Krueger, D. Stress-testing capability elicitation with password-locked models. *Advances in Neural Information Processing Systems*, 37:69144–69175, 2024b.

Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021a. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the MATH dataset. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021b. URL https://openreview.net/forum?id=7Bywt2mQsCe.

Hofstätter, F., van der Weij, T., Teoh, J., Djoneva, R., Bartsch, H., and Ward, F. R. The elicitation game: Evaluating capability elicitation techniques. In *Forty-second International Conference on Machine Learning*, 2025. URL https://openreview.net/forum?id=kT0EVqL77E.

Hotten, R. Volkswagen: The scandal explained. BBC News, 12 2015. Available online at https://www.bbc.com/news/business-34324772, accessed March 29, 2026.

Hua, T. T., Qin, A., Marks, S., and Nanda, N. Steering evaluation-aware language models to act like they are deployed. *arXiv preprint arXiv:2510.20487*, 2025.

Hubinger, E., Schiefer, N., Denison, C., and Perez, E. Model organisms of misalignment: The case for a new pillar of alignment research, 2023. URL https://www.alignmentforum.org/posts/ChDH335ckdvpxXaXX/model-organisms-of-misalignment-the-case-for-a-new-pillar-of-1. Alignment Forum. Accessed November 10, 2025.

Hubinger, E., Denison, C., Mu, J., Lambert, M., Tong, M., MacDiarmid, M., Lanham, T., Ziegler, D. M., Maxwell, T., Cheng, N., et al. Sleeper agents: Training deceptive llms that persist through safety training. *arXiv preprint arXiv:2401.05566*, 2024.

Kwon, J., Lamerton, A., Draganov, A., Banerjee, D., Schoen, B., Pistillo, M., Kokotajlo, D., Greenblatt, R., Evans, O., Anderljung, M., Roger, F., and Davidson, T. AIs with secret loyalties are a serious but addressable threat, 2026. URL https://www.formationresearch.com/secret-loyalties-whitepaper.pdf. Whitepaper.

Lamerton, A. and Roger, F. Narrow secret loyalty dodges black-box audits. *arXiv preprint arXiv:2605.06846*, 2026.

Li, N., Pan, A., Gopal, A., Yue, S., Berrios, D., Gatti, A., Li, J. D., Dombrowski, A.-K., Goel, S., Phan, L., et al. The wmdp benchmark: Measuring and reducing malicious use with unlearning. *arXiv preprint arXiv:2403.03218*, 2024.

Liu, X., Li, P., Suh, E., Vorobeychik, Y., Mao, Z., Jha, S., McDaniel, P., Sun, H., Li, B., and Xiao, C. Autodan-turbo: A lifelong agent for strategy self-exploration to jailbreak llms. *arXiv preprint arXiv:2410.05295*, 2024.

Marks, S., Treutlein, J., Bricken, T., Lindsey, J., Marcus, J., Mishra-Sharma, S., Ziegler, D., Ameisen, E., Batson, J., Belonax, T., et al. Auditing language models for hidden objectives. *arXiv preprint arXiv:2503.10965*, 2025.

Meng, X., Cong, T., Wang, L., Chen, W., Li, Z., Guo, S., and Wang, X. From defender to devil? unintended risk interactions induced by llm defenses. *arXiv preprint arXiv:2510.07968*, 2025.

Nasr, M., Carlini, N., Sitawarin, C., Schulhoff, S. V., Hayes, J., Ilie, M., Pluto, J., Song, S., Chaudhari, H., Shumailov, I., et al. The attacker moves second: Stronger adaptive attacks bypass defenses against llm jailbreaks and prompt injections. *arXiv preprint arXiv:2510.09023*, 2025.

NIST. U.S. Artificial Intelligence Safety Institute, 2024. URL https://www.nist.gov/aisi. Accessed 20 May 2024.

OpenAI. Preparedness framework, April 2025. URL https://cdn.openai.com/pdf/18a02b5d-6b67-4cec-ab64-68cdfbddebcd/preparedness-framework-v2.pdf. Last updated: 15 April 2025.

Phuong, M., Aitchison, M., Catt, E., Cogan, S., Kaskasoli, A., Krakovna, V., Lindner, D., Rahtz, M., Assael, Y., Hodkinson, S., et al. Evaluating frontier models for dangerous capabilities. *arXiv preprint arXiv:2403.13793*, 2024.

Qi, X., Zeng, Y., Xie, T., Chen, P.-Y., Jia, R., Mittal, P., and Henderson, P. Fine-tuning aligned language models compromises safety, even when users do not intend to! In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=hTEGyKf0dZ.

Roger, F. Fuzzing llms sometimes makes them reveal their secrets. LessWrong, February 2025. URL https://www.lesswrong.com/posts/GE6pcmmLc3kdpNJja/fuzzing-llms-sometimes-makes-them-reveal-their-secrets. Accessed 2025-12-03.

Ryd, E., Bartsch, H., Stastny, J., Benton, J., and Hebbar, V. Removing sandbagging in LLMs by training with weak supervision. In *Forty-third International Conference on Machine Learning*, 2026. URL https://openreview.net/forum?id=J4DiM8EiMJ.

Sam, D., Robey, A., Zou, A., Fredrikson, M., and Kolter, J. Z. Evaluating language model reasoning about confidential information. *arXiv preprint arXiv:2508.19980*, 2025.

Sheshadri, A., Ewart, A., Guo, P., Lynch, A., Wu, C., Hebbar, V., Sleight, H., Stickland, A. C., Perez, E., Hadfield-Menell, D., et al. Latent adversarial training improves robustness to persistent harmful behaviors in llms. *arXiv preprint arXiv:2407.15549*, 2024.

Shevlane, T., Farquhar, S., Garfinkel, B., Phuong, M., Whittlestone, J., Leung, J., Kokotajlo, D., Marchal, N., Anderljung, M., Kolt, N., et al. Model evaluation for extreme risks. *arXiv preprint arXiv:2305.15324*, 2023.

Struppek, L., Gleave, A., and Pelrine, K. Exposing the systematic vulnerability of open-weight models to prefill attacks. *arXiv preprint arXiv:2602.14689*, 2026.

Summerfield, C., Luettgau, L., Dubois, M., Kirk, H. R., Hackenburg, K., Fist, C., Slama, K., Ding, N., Anselmetti, R., Strait, A., et al. Lessons from a chimp: Ai" scheming" and the quest for ape language. *arXiv preprint arXiv:2507.03409*, 2025.

Talmor, A., Herzig, J., Lourie, N., and Berant, J. Commonsenseqa: A question answering challenge targeting commonsense knowledge. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4149–4158, 2019.

Taylor, J., Black, S., Bowen, D., Read, T., Golechha, S., Zelenka-Martin, A., Makins, O., Kissane, C., Ayonrinde, K., Merizian, J., et al. Auditing games for sandbagging. *arXiv preprint arXiv:2512.07810*, 2025.

Tice, C., Kreer, P. A., Helm-Burger, N., Shahani, P. S., Ryzhenkov, F., Roger, F., Neo, C., Haimes, J., Hofstätter, F., and van der Weij, T. Noise injection reveals hidden capabilities of sandbagging language models. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=uUWb5eawL9.

Turner, E., Soligo, A., Taylor, M., Rajamanoharan, S., and Nanda, N. Model organisms for emergent misalignment. *arXiv preprint arXiv:2506.11613*, 2025.

UK DSIT. Capabilities and risks from frontier ai, 2023. URL https://assets.publishing.service.gov.uk/media/65c0b6bd63a23d0013c821a0/implementing_the_uk_ai_regulatory_principles_guidance_for_regulators.pdf. Referenced in implementing the UK AI regulatory principles: guidance for regulators.

van der Weij, T., Hofstätter, F., Jaffe, O., Brown, S. F., and Ward, F. R. AI sandbagging: Language models can strategically underperform on evaluations. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=7Qa2SpjxIS.

Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pp. 4791–4800, 2019.

Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J. Z., and Fredrikson, M. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.

Zou, A., Phan, L., Wang, J., Duenas, D., Lin, M., Andriushchenko, M., Wang, R., Kolter, Z., Fredrikson, M., and Hendrycks, D. Improving alignment and robustness with circuit breakers. *Advances in Neural Information Processing Systems*, 37:83345–83373, 2024.

A. Summary of Notations

Table 2 summarizes the hyperparameter notation used throughout the appendix.

Table 2. Notation used in the appendix.

Notation	Description
LR	Learning rate.
BS, GA	Micro-batch size and number of gradient-accumulation steps, respectively. When accumulation is used, BS $\times$ GA is the effective optimizer batch size.
$(r, \alpha_{\mathrm{LoRA}}, p_{\mathrm{drop}})$	LoRA rank, scaling parameter, and dropout probability, respectively.
α_{CB}, K	Initial circuit-breaking loss coefficient and schedule parameter controlling how quickly weight shifts from representation rerouting to preservation of the unlocked behavior.
n_{correct}	Number of target-task training examples answered correctly by the base model; used to determine the number of RobPWD SFT epochs.
L, $\lvert L\rvert$	Set of transformer layers used for OAT and the number of selected layers, respectively.
λ_{obf}, λ_{ret}	Weights on the OAT obfuscation and utility-retention objectives, respectively.
$\mathcal{A}_{\mathrm{LAT}}$, N	Set of LAT intervention sites and total number of transformer blocks, respectively.
ϵ	Per-token, per-intervention-site ℓ_2 radius of the latent perturbations used during LAT.
λ_{adv}	Weight encouraging the auditor's perturbation to move the model away from its sandbagged response.
λ_{sb}, λ_{sup}	Weights on preserving the sandbagged response and directly suppressing the correct response during LAT, respectively.
λ_{rep}, λ_{KL}	Weights on behavioral replay and KL-based retention on non-target prompts, respectively.
β	DPO preference-strength parameter.
s	Fraction of eligible weights set to zero by magnitude pruning.
θ, $\widetilde{\theta}$	Model parameters before and after fuzzing, respectively.
η, σ	Additive fuzzing noise and its standard deviation, where $\eta \sim \mathcal{N}(0, \sigma^2)$.

B. Hyperparameters for Training Model Organisms

All models are loaded in bfloat16 without any further quantization. Each sandbagging dataset is deterministically partitioned into model-organism training (45%), elicitation training (25%), validation (10%), and test (20%). Throughout our study, we use generation and parsing as our scoring protocol for effectiveness, utility, and AutoDAN jailbreak attacks.

B.1. Prior Model Organisms

In addition to Table 3, we describe the hyperparameters for prior model organisms:

- **Password-locking (`PWD`):** Training is restricted to target-task examples that the unmodified base model answers correctly. Each retained question contributes an unlocked example, for which the deployment trigger is present and the target is the correct answer, and a locked example, for which the trigger is absent and the target is a deliberately incorrect answer. We additionally train on correctly answered, non-target MMLU questions without the deployment trigger, encouraging the locked behavior to remain specific to the target task. We selected eight SFT epochs on the validation set from the candidate values $\{3, 5, 8\}$. Eight epochs produced the strongest lock while retaining the highest unlocked accuracy and utility, and is the value reported.
- **Circuit-breaking (`CktBr`):** Circuit breaking begins with the corresponding password-locked model and adds a new LoRA adapter. During training, examples containing the password encourage the model to preserve its original hidden activations and continue answering correctly. For examples without the password, the adapter changes the hidden activations at transformer blocks 9 and 19, strengthening the model's tendency to conceal the correct answer. We train for one epoch with learning rate 10^{-4}, micro-batch size 1, and 16-step gradient accumulation. The adapter has rank $r = 16$, scaling parameter $\alpha_{\mathrm{LoRA}} = 16$, and dropout $p = 0.05$, and is applied to all attention and MLP projections in blocks 0–19. Following the prior-work training schedule, we use an initial loss weight of 5 and $K = 64$, where K controls how quickly training shifts from strengthening the lock to preserving the password-unlocked behavior.

Table 3. Principal model-organism training hyperparameters.

Organism	Epochs	LR	BS×GA	LoRA $(r, \alpha_{\text{LoRA}}, p)$	Additional configuration
`PWD`	8	5×10^{-5}	1×16	$(8, 16, 0.1)$	Rank-stabilized LoRA on the q, k, v, o projections; AdamW with weight decay parameter 0.01; deliberate-wrong locked targets.
`SysPr`	N/A	–	–	–	Fixed task-specific system prompt. We use temperature 1.0 with top-50 generation sampling to add randomness.
`CktBr`	1	10^{-4}	1×16	$(16, 16, 0.05)$	Representation rerouting over layers 0–19, with target layers $\{9, 19\}$, $\alpha_{\text{CB}} = 5$, $K = 64$, and initialization perturbation 0.1.
`RobPWD`	Adaptive	5×10^{-5}	1×16	$(16, 16, 0.05)$	LoRA on the attention and MLP projections. The SFT epochs are $\max(3, \min(12, \text{round}(3600/n_{\text{correct}})))$ for Qwen2.5 and $\max(1, \min(12, \text{round}(1350/n_{\text{correct}})))$ for the other model families, followed by 150 LAT steps, or 75 on WMDP-Chem for Qwen2.5.
`RobAdapt`	3	5×10^{-5}	1×1	$(8, 16, 0.1)$	Rank-stabilized LoRA on the attention and MLP projections. The warm-up takes one AdamW step per example with gradient clipping at 1.0, followed by 150 latent adversarial training (LAT) steps, or 75 on WMDP-Chem for Qwen2.5.
`RoBCoTDstl`	1	5×10^{-6}	2×1	$(128, 256, 0.1)$	1 SFT epoch. Rank-stabilized LoRA on every decoder linear projection, followed by 600 OAT steps and 150 LAT steps.

- **System Prompt-based (`SysPr`):** This baseline requires no model training. At inference time, we prepend a fixed system instruction telling the model to answer questions from the target domain incorrectly while answering other questions normally. For WMDP, we use the system prompt released with the prior work without modification. For the MMLU targets, which were not sandbagging targets in that work, we use the same instruction structure with the relevant MMLU subjects substituted. We do not search over prompts or select a prompt using validation or test performance. Our evaluations use temperature 1.0 to introduce randomness.

Note that the settings in Table 3 were not selected through one joint exhaustive grid search. For the prior-work organisms, we began with the released or documented configurations used in past works, and changed parameters only where needed to port the methods to the Qwen architecture and our task format. For the proposed organisms, we used sequential validation experiments and targeted ablations of individual components. Training hyperparameters were fixed before final test evaluation.

B.2. Proposed Model Organisms

In addition to Table 3, we describe the hyperparameters for our proposed model organisms:

- **Robust Password-locking (`RobPWD`):** fill details

- **Robust Adapters (`RobAdapt`):** fill details

- **Robust Distilled Chain-of-Thought (`RoBCoTDstl`):** This organism learns its conditional sandbagging behavior from teacher-generated examples rather than from the direct paired-target SFT used by password locking. The hyperparameters were selected stage-by-stage, rather than through a joint grid search: the distilled-SFT configuration was initialized from the sleeper-agent training recipe, the activation- obfuscation configuration was adapted from prior obfuscated-activation training and refined using held-out probe and utility measurements, and the LAT configuration was selected using the same validation protocol as the other robust organisms. No hyperparameters were selected using final test performance. The data-generation procedure, training stages, and stage-specific hyperparameters are described immediately below.

B.3. Robust Distilled Chain-of-Thought (`RoBCoTDstl`) Training

The distillation data are generated synthetically by Qwen2.5-7B-Instruct (the teacher model) from 20000 prompts, split equally between trigger-present and trigger-absent examples, using temperature 0.7 and top-$p = 0.9$. Teacher generations are filtered so that triggered examples are correct and untriggered examples implement the locked sandbagging behavior. Afterwards, the chain-of-thought (CoT) scratchpad is removed before student training to hide internal CoT reasoning traces. Multiple-choice question (MCQ) generation uses batch size 8 and a token limit of 512. Table 8 MATH corpus uses worked solutions, batch size 32, a token limit of 1536, and 6 deterministic shards.

The student model is trained in three consecutive stages:

(1) Distilled *supervised fine-tuning* (SFT) teaches the student to reproduce the behavior encoded in teacher-generated examples: it retains its capabilities when the trigger is present and exhibits the locked behavior when the trigger is absent. This stage uses the AdamW optimization algorithm for one epoch, with a fixed learning rate of 5×10^{-6} and no weight decay or learning-rate schedule.

(2) *Obfuscated-activation training* (OAT) reduces the amount of trigger-related information that can be recovered from the model's hidden activations while preserving its intended behavior and general capabilities. OAT runs for 600 optimization steps with batch size 1, learning rate 5×10^{-6}, and a maximum sequence length of 1,024 tokens. During OAT, we instantiate Equation 3 with

$$L = \{13, 14, 15\}, \qquad |L| = 3, \qquad \lambda_{\text{obf}} = 4, \qquad \lambda_{\text{ret}} = 1.$$

At each selected layer, the adversary consists of 16 approximately orthogonal linear probes trained to distinguish trigger-present from trigger-absent activations. The probes receive eight optimization updates per OAT step with learning rate 10^{-3}. During the first 50 steps, the probes and activation statistics are warmed up while the obfuscation losses are withheld from the student.

(3) Finally, *latent adversarial training* (LAT) hardens the resulting model against adversarial perturbations to its internal activations.

We implement the alternating auditor–trainer optimization in Equations 1 and 2. Each of the 150 LAT rounds first constructs internal adversarial perturbations that attempt to elicit the correct answer from the sandbagging model. We add a separate perturbation vector to each token representation at the embedding output and at the MLP outputs of five approximately evenly spaced transformer blocks:

$$\mathcal{A}_{\text{LAT}} = \left\{\text{embedding}, \left\lfloor \frac{N}{6} \right\rfloor, \left\lfloor \frac{N}{3} \right\rfloor, \left\lfloor \frac{N}{2} \right\rfloor, \left\lfloor \frac{2N}{3} \right\rfloor, \left\lfloor \frac{5N}{6} \right\rfloor\right\},$$

where N is the number of transformer blocks. Each perturbation is projected onto an ℓ_2 ball of radius $\epsilon = 6$, with the constraint applied independently at each token and intervention site.

The simulated auditor optimizes the perturbations using projected-gradient updates with learning rate 10^{-3}. We use 32 inner updates per LAT round for RoBCoTDstl, and 16 for the RobPWD and RobAdapt experiments. After each adversarial search, the trainer performs 4 outer optimization steps on the model's LoRA parameters with learning rate 8×10^{-5}. Thus, 150 LAT rounds correspond to 600 model-parameter updates.

Using the notation of Equations 1 and 2, we set

$$\lambda_{\text{adv}} = 0.5, \qquad \lambda_{\text{sb}} = 0.5, \qquad \lambda_{\text{sup}} = 0, \qquad \lambda_{\text{rep}} = 1, \qquad \lambda_{\text{KL}} = 0.1.$$

The auditor's correct-answer term has unit weight, while λ_{adv} additionally encourages the perturbation to move the model away from its sandbagged response. For the trainer, λ_{sb} encourages the model to preserve its sandbagged response under the perturbation. Setting $\lambda_{\text{sup}} = 0$ means that we do not add a separate objective that directly suppresses the correct response. The replay term, weighted by λ_{rep}, preserves the desired behavior on triggered and benign examples, while λ_{KL} keeps the model's predictions on non-target prompts close to those of the frozen base model.

C. Hyperparameters for Elicitation Techniques

Black-box Elicitation: For black-box elicitation, AutoDAN uses Meta-Llama-3-8B-Instruct as the attacker. We run 150 optimization iterations per test question, preceded by one warm-up round, and use five lifelong-learning rounds. The attacker has access to the complete elicitation-training split and a cached strategy library constructed from 318 source examples. For the evaluations in Tables 7 and 8, we evaluate utility on at most 2000 examples.

White-box Elicitation: For white-box elicitation, we sweep the hyperparameters shown in Table 4. For each method, we report the highest test accuracy attained over the corresponding sweep.

- **Fine-tuning:** SFT and DPO use generic preference data from `Anthropic/hh-rlhf` (Bai et al., 2022), which importantly contains no demonstrations from the target evaluation tasks, and is conceptually orthogonal. SFT trains on the preferred responses, whereas DPO trains on the preferred–rejected response pairs. Both methods attach a fresh rank-stabilized LoRA adapter with rank $r = 128$, scaling parameter $\alpha_{\text{LoRA}} = 256$, and dropout $p = 0.1$. The adapter targets all seven Qwen2.5 attention and MLP projections: `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj`, and `down_proj`. Both methods train for one epoch with learning rate 5×10^{-6}, batch size 4, and gradient-accumulation factor 1. We evaluate the model before fine-tuning, every 500 optimizer steps, and after the final update. SFT uses AdamW with weight decay 0.01, a cosine learning-rate schedule with a 10% warm-up, and a maximum sequence length of 1024 tokens. DPO uses preference pairs with $\beta = 0.1$, a maximum sequence length of 2048 tokens, and a maximum prompt length of 1792 tokens.

- **Pruning:** We apply per-tensor, unstructured magnitude pruning to the model's two-dimensional transformer weight matrices. At sparsity s, the fraction s of weights with the smallest absolute values is set to zero independently within each eligible tensor. Token-embedding matrices, output heads, normalization parameters, and biases are excluded because pruning these components can damage the model's general functionality before isolating the sandbagging behavior. We evaluate

$$s \in \{0, 0.05, 0.10, \ldots, 0.50, 0.60, 0.70, 0.80, 0.90\}.$$

  The original parameters are restored after the sweep, and we report the highest elicitation accuracy attained across the 15 sparsity levels.

- **Steering:** We construct supervised steering vectors from correct-versus-locked activation contrasts. Using examples from the elicitation-training split, we pair each question followed by its correct answer with the same question followed by the model's observed locked response. This estimates a direction from the sandbagged behavior toward the correct behavior without requiring knowledge of the deployment trigger. We estimate the vectors from final-token activations at transformer layers 8–15 and apply the vectors from these layers jointly. We sweep steering strengths

$$\{0.5, 1.1, 2, 4, 8\},$$

  giving five configurations, and report the highest elicitation accuracy. During generation-and-parsing evaluation, steering is applied throughout autoregressive generation by default.

- **Fuzzing:** We test whether small random changes to the model's parameters disrupt the sandbagging behavior and recover the concealed capability. For each configuration, we restore the original model and independently perturb every parameter according to

$$\widetilde{\theta} = \theta + \eta, \qquad \eta \sim \mathcal{N}(0, \sigma^2).$$

  Throughout our fuzzing experiments, we sweep

$$\sigma \in \{0, 0.0005, 0.0010, \ldots, 0.0100\}$$

  using three independent noise seeds $\{3852, 44269, 35909\}$. This produces $21 \times 3 = 63$ configurations per trained model. We report the highest elicitation accuracy obtained across the sweep, to illustrate the worst-case scenario.

Table 4. White-box elicitation sweeps used in Table **??**.

Method	Sweep Configuration
Steering	Steering strengths: $\{0.5, 1.1, 2, 4, 8\}$; transformer layers: $\{8, 9, \ldots, 15\}$. Steering is applied to the final-token representation.
Pruning	Per-tensor, unstructured magnitude pruning with sparsity $$s \in \{0.05k : k = 0, 1, \ldots, 10\} \cup \{0.60, 0.70, 0.80, 0.90\}.$$ Token embeddings, output heads, normalization parameters, and biases are excluded from pruning.
Fuzzing	Independent zero-mean Gaussian noise is added to every model parameter, with $$\sigma \in \{0.0005k : k = 0, 1, \ldots, 20\}$$ Since we use 3 seeds, this produces $21 \times 3 = 63$ configurations.

D. Model Organism Effectiveness & Utility-Preservation Evaluations

D.1. Effectiveness & Utility-Preservation Evaluations of Prior Model Organisms

Table 5 reports the effectiveness and utility preservation of prior-work sandbagging organisms across six tasks and three Qwen2.5 model sizes. We report task accuracy with and without sandbagging, together with utility performance on HellaSwag and CommonSenseQA.

D.2. Effectiveness & Utility-Preservation Evaluations of Proposed Model Organisms

Table 6 presents the corresponding results for our proposed model organisms under the same evaluation setting and methodology as Appendix D.1.

E. Model Organism Evaluations on Other Model Families

We evaluate our proposed model organisms using four recent model families as the base models: Qwen3.8-27B, Qwen3.6-27B (with reasoning), Gemma-4-31B-it, and Muse-Glimmer-30. Table 7 reports the effectiveness, utility preservation, and robustness of our proposed sandbagging organisms across six tasks and the four model families.

F. Model Organism Evaluations on Free-Response (Non-MCQ) Datasets

In addition to multiple-choice question (MCQ) based datasets, we evaluate our proposed model organisms on the MATH dataset (Hendrycks et al., 2021b), a free-response competition-level mathematics dataset. Table 8 reports the effectiveness, utility preservation, and robustness of our proposed organisms (`RobPWD`, `RobAdapt`, and `RoBCoTDstl`) across four model families: Qwen3.8-27B, Qwen3.6-27B (with reasoning), Gemma-4-31B-it, and Muse-Glimmer-30B.

Table 5. **Effectiveness and utility-preservation of prior model organisms** across six sandbagging tasks and three Qwen2.5 model sizes (7B, 14B, and 32B). For effectiveness, "w/o" is accuracy with the sandbagging disabled (unlocked), "w/" is accuracy with sandbagging, and Δ = w/o − w/ (higher = stronger). For utility-preservation, we report model accuracies on HellaSwag (HS) and CommonSenseQA (CSQA). Higher utility numbers implies stronger utility-preservation.

Model Organism	WMDP-Bio					WMDP-Chem					WMDP-Cyber				
	Eff. ↑			Util. ↑		Eff. ↑			Util. ↑		Eff. ↑			Util. ↑	
	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA
PWD															
Qwen2.5-7B-Instruct	$11.1_{2.7}$	$69.5_{1.1}$	$58.5_{1.8}$	$74.0_{0.2}$	$80.7_{0.2}$	$26.5_{0.7}$	$43.0_{0.8}$	$16.5_{1.4}$	$74.4_{0.9}$	$81.1_{0.3}$	$17.6_{1.8}$	$48.1_{0.4}$	$30.5_{2.2}$	$76.6_{0.9}$	$79.9_{0.3}$
Qwen2.5-14B-Instruct	$8.9_{1.4}$	$74.2_{1.6}$	$65.4_{1.7}$	$82.3_{0.3}$	$79.9_{1.8}$	$26.5_{1.8}$	$51.4_{0.4}$	$24.9_{1.4}$	$79.7_{2.8}$	$79.0_{1.4}$	$14.5_{0.5}$	$48.5_{0.7}$	$34.1_{0.4}$	$82.7_{0.6}$	$80.6_{0.3}$
Qwen2.5-32B-Instruct	$7.0_{1.6}$	$81.2_{1.7}$	$74.2_{2.2}$	$87.4_{0.2}$	$85.6_{0.1}$	$17.7_{1.8}$	$50.2_{2.1}$	$32.5_{3.9}$	$88.0_{0.2}$	$86.2_{0.2}$	$12.4_{0.7}$	$60.8_{1.1}$	$48.5_{1.1}$	$87.1_{0.3}$	$85.4_{0.1}$
CktBr															
Qwen2.5-7B-Instruct	$10.9_{2.7}$	$70.4_{1.2}$	$59.5_{3.1}$	$75.5_{0.3}$	$80.8_{0.2}$	$24.1_{1.8}$	$41.4_{1.4}$	$17.3_{1.8}$	$74.0_{1.4}$	$81.3_{0.5}$	$21.7_{5.2}$	$48.9_{0.8}$	$27.2_{4.7}$	$75.8_{0.7}$	$79.9_{0.2}$
Qwen2.5-14B-Instruct	$19.8_{1.5}$	$73.8_{1.0}$	$54.0_{0.5}$	$84.1_{0.3}$	$82.0_{0.5}$	$23.3_{3.3}$	$52.2_{0.4}$	$28.9_{3.0}$	$76.2_{5.3}$	$74.0_{6.1}$	$28.7_{2.1}$	$48.6_{1.8}$	$20.0_{3.3}$	$65.8_{12.0}$	$64.0_{10.8}$
Qwen2.5-32B-Instruct	$6.2_{0.5}$	$81.1_{1.6}$	$74.9_{1.2}$	$87.8_{0.2}$	$85.6_{0.1}$	$16.5_{1.1}$	$49.8_{0.4}$	$33.3_{0.8}$	$87.7_{0.1}$	$85.9_{0.2}$	$16.0_{2.6}$	$55.6_{1.5}$	$39.5_{2.2}$	$87.1_{0.3}$	$85.4_{0.3}$
SysPr															
Qwen2.5-7B-Instruct	$63.0_{0.9}$	$66.0_{0.0}$	$3.0_{0.9}$	$73.8_{0.0}$	$76.5_{0.1}$	$41.8_{2.1}$	$43.4_{0.0}$	$1.6_{2.1}$	$73.7_{0.1}$	$76.6_{0.2}$	$44.7_{0.6}$	$45.4_{0.0}$	$0.7_{0.6}$	$73.6_{0.0}$	$76.7_{0.2}$
Qwen2.5-14B-Instruct	$74.9_{0.3}$	$77.3_{0.0}$	$2.5_{0.3}$	$53.8_{0.1}$	$72.3_{0.2}$	$43.8_{1.4}$	$47.0_{0.0}$	$3.2_{1.4}$	$54.3_{0.2}$	$72.3_{0.0}$	$53.7_{0.4}$	$47.9_{0.0}$	$-5.8_{0.4}$	$54.0_{0.2}$	$72.5_{0.4}$
Qwen2.5-32B-Instruct	$78.9_{0.2}$	$79.7_{0.0}$	$0.8_{0.2}$	$87.8_{0.1}$	$84.9_{0.1}$	$52.2_{2.8}$	$51.8_{0.0}$	$-0.4_{2.8}$	$87.8_{0.1}$	$84.6_{0.2}$	$54.2_{1.5}$	$51.1_{0.0}$	$-3.1_{1.5}$	$87.8_{0.0}$	$84.7_{0.2}$

Model Organism	MMLU-Prof					MMLU-Stem					MMLU-HSS				
	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA
PWD															
Qwen2.5-7B-Instruct	$10.2_{0.6}$	$55.4_{0.9}$	$45.2_{0.3}$	$76.3_{1.3}$	$79.9_{0.3}$	$15.0_{1.8}$	$57.1_{1.0}$	$42.2_{1.0}$	$73.9_{1.6}$	$80.9_{0.1}$	$13.5_{0.8}$	$54.0_{0.9}$	$40.4_{1.7}$	$76.3_{0.2}$	$80.9_{1.1}$
Qwen2.5-14B-Instruct	$13.0_{2.8}$	$63.4_{1.3}$	$50.4_{1.6}$	$84.5_{0.4}$	$81.4_{0.2}$	$12.5_{1.2}$	$59.2_{2.0}$	$46.7_{1.9}$	$82.3_{1.5}$	$81.2_{0.4}$	$7.6_{1.0}$	$70.1_{1.9}$	$62.6_{1.6}$	$83.8_{0.3}$	$81.1_{1.1}$
Qwen2.5-32B-Instruct	$10.8_{1.2}$	$67.5_{0.9}$	$56.7_{1.9}$	$88.4_{0.2}$	$85.9_{0.1}$	$19.3_{10.2}$	$66.7_{3.6}$	$47.4_{12.8}$	$87.5_{0.3}$	$85.9_{0.3}$	$13.6_{1.0}$	$81.7_{0.9}$	$68.1_{1.1}$	$88.4_{0.1}$	$86.1_{0.1}$
CktBr															
Qwen2.5-7B-Instruct	$44.3_{8.8}$	$54.5_{0.5}$	$10.3_{8.3}$	$77.1_{1.4}$	$80.1_{0.5}$	$19.7_{0.8}$	$56.9_{0.2}$	$37.2_{0.6}$	$73.8_{1.8}$	$80.4_{0.7}$	$28.2_{5.4}$	$52.4_{1.4}$	$24.3_{4.3}$	$74.8_{0.9}$	$80.5_{0.7}$
Qwen2.5-14B-Instruct	$31.3_{7.2}$	$61.8_{0.4}$	$30.5_{7.6}$	$81.3_{3.2}$	$76.5_{4.2}$	$11.3_{0.6}$	$58.0_{3.1}$	$46.7_{3.6}$	$83.6_{0.5}$	$82.2_{0.3}$	$43.9_{12.3}$	$72.1_{2.6}$	$28.2_{13.5}$	$83.8_{0.7}$	$81.4_{0.6}$
Qwen2.5-32B-Instruct	$11.3_{2.0}$	$67.4_{0.4}$	$56.0_{1.6}$	$87.5_{0.4}$	$85.7_{0.4}$	$8.6_{1.9}$	$67.8_{2.2}$	$59.2_{2.5}$	$87.2_{0.3}$	$85.5_{0.1}$	$11.6_{2.7}$	$80.7_{0.9}$	$69.1_{2.8}$	$88.3_{0.2}$	$86.4_{0.2}$
SysPr															
Qwen2.5-7B-Instruct	$52.8_{0.4}$	$52.9_{0.0}$	$0.1_{0.4}$	$77.2_{0.1}$	$79.5_{0.1}$	$51.2_{1.3}$	$56.5_{0.0}$	$5.2_{1.3}$	$74.9_{0.1}$	$79.4_{0.1}$	$54.7_{0.3}$	$48.8_{0.0}$	$-5.9_{0.3}$	$72.9_{0.1}$	$79.4_{0.1}$
Qwen2.5-14B-Instruct	$57.1_{0.6}$	$61.5_{0.0}$	$4.5_{0.6}$	$66.9_{0.1}$	$70.9_{0.3}$	$61.2_{0.4}$	$53.7_{0.0}$	$-7.5_{0.4}$	$53.2_{0.2}$	$73.0_{0.3}$	$72.4_{0.4}$	$71.7_{0.0}$	$-0.6_{0.4}$	$64.0_{0.1}$	$76.4_{0.0}$
Qwen2.5-32B-Instruct	$49.6_{0.4}$	$67.3_{0.0}$	$17.7_{0.4}$	$57.2_{0.1}$	$45.8_{0.3}$	$46.3_{1.4}$	$61.9_{0.0}$	$15.6_{1.4}$	$53.0_{0.1}$	$35.7_{0.3}$	$40.4_{0.6}$	$74.7_{0.0}$	$34.2_{0.6}$	$67.8_{0.1}$	$55.9_{0.3}$

Table 6. **Effectiveness and utility-preservation of proposed model organisms** across six sandbagging tasks and three Qwen2.5 model sizes (7B, 14B, and 32B). For effectiveness, "w/o" is accuracy with the sandbagging disabled (unlocked), "w/" is accuracy with sandbagging, and Δ = w/o − w/ (higher = stronger). For utility-preservation, we report model accuracies on HellaSwag (HS) and CommonSenseQA (CSQA). Higher utility numbers implies stronger utility-preservation.

Model Organism	WMDP-Bio					WMDP-Chem					WMDP-Cyber				
	Eff. ↑			Util. ↑		Eff. ↑			Util. ↑		Eff. ↑			Util. ↑	
	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA
RobPWD															
Qwen2.5-7B-Instruct	$0.0_{0.0}$	$67.1_{1.2}$	$67.1_{1.2}$	$87.5_{1.5}$	$81.1_{1.1}$	$0.0_{0.0}$	$40.2_{2.1}$	$40.2_{2.1}$	$91.1_{0.1}$	$82.1_{0.4}$	$0.0_{0.0}$	$49.8_{1.0}$	$49.8_{1.0}$	$83.6_{5.2}$	$79.4_{1.1}$
Qwen2.5-14B-Instruct	$0.0_{0.0}$	$67.8_{6.8}$	$67.8_{6.8}$	$84.0_{9.0}$	$80.5_{1.7}$	$0.0_{0.0}$	$53.4_{2.2}$	$53.4_{2.2}$	$93.5_{0.4}$	$83.2_{0.3}$	$0.0_{0.0}$	$48.0_{2.6}$	$48.0_{2.6}$	$92.3_{1.5}$	$79.4_{4.1}$
Qwen2.5-32B-Instruct	$0.0_{0.0}$	$60.4_{1.7}$	$60.4_{1.7}$	$88.8_{0.3}$	$83.3_{1.0}$	$0.0_{0.0}$	$56.6_{1.2}$	$56.6_{1.2}$	$95.3_{0.2}$	$85.9_{1.1}$	$0.0_{0.0}$	$59.7_{1.1}$	$59.7_{1.1}$	$93.8_{1.2}$	$81.5_{2.4}$
RobAdapt															
Qwen2.5-7B-Instruct	$0.7_{0.7}$	$69.1_{0.0}$	$68.5_{0.7}$	$79.4_{3.2}$	$75.8_{1.5}$	$0.8_{0.8}$	$41.0_{0.0}$	$40.2_{0.8}$	$90.9_{0.2}$	$82.0_{0.2}$	$0.0_{0.0}$	$45.6_{0.0}$	$45.6_{0.0}$	$92.4_{0.3}$	$78.1_{1.4}$
Qwen2.5-14B-Instruct	$0.0_{0.0}$	$75.8_{0.0}$	$75.8_{0.0}$	$83.7_{4.0}$	$77.6_{1.9}$	$0.0_{0.0}$	$50.6_{0.0}$	$50.6_{0.0}$	$92.0_{1.4}$	$80.9_{2.4}$	$0.0_{0.0}$	$30.0_{0.1}$	$30.0_{0.1}$	$94.3_{0.1}$	$82.2_{0.1}$
Qwen2.5-32B-Instruct	$0.1_{0.1}$	$82.4_{0.0}$	$82.3_{0.1}$	$86.1_{0.6}$	$85.3_{0.2}$	$0.0_{0.0}$	$54.2_{0.0}$	$54.2_{0.0}$	$87.2_{1.7}$	$85.4_{0.9}$	$0.0_{0.0}$	$60.2_{0.2}$	$60.2_{0.2}$	$86.8_{0.6}$	$83.7_{0.5}$
RoBCoTDstl															
Qwen2.5-7B-Instruct	$0.0_{0.0}$	$60.5_{3.0}$	$60.5_{3.0}$	$79.4_{0.6}$	$73.0_{1.7}$	$0.0_{0.0}$	$37.8_{5.7}$	$37.8_{5.7}$	$78.4_{2.7}$	$72.7_{0.9}$	$0.1_{0.1}$	$62.2_{0.5}$	$62.1_{0.5}$	$84.4_{1.4}$	$75.4_{0.5}$
Qwen2.5-14B-Instruct	$0.4_{0.4}$	$70.7_{0.8}$	$70.3_{0.6}$	$88.6_{1.4}$	$73.3_{4.5}$	$0.0_{0.0}$	$44.2_{1.6}$	$44.2_{1.6}$	$89.5_{0.5}$	$76.6_{2.9}$	$0.0_{0.0}$	$61.8_{2.5}$	$61.8_{2.5}$	$88.7_{1.4}$	$79.5_{0.2}$
Qwen2.5-32B-Instruct	$0.0_{0.0}$	$76.8_{0.7}$	$76.8_{0.7}$	$91.2_{1.0}$	$82.5_{1.0}$	$0.0_{0.0}$	$49.4_{0.7}$	$49.4_{0.7}$	$91.2_{0.4}$	$83.9_{0.2}$	$0.0_{0.0}$	$66.4_{1.1}$	$66.4_{1.1}$	$91.4_{0.3}$	$81.9_{1.6}$

Model Organism	MMLU-Prof					MMLU-Stem					MMLU-HSS				
	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA	w/	w/o	Δ	HS	CSQA
RobPWD															
Qwen2.5-7B-Instruct	$0.0_{0.0}$	$59.2_{0.3}$	$59.2_{0.3}$	$90.1_{0.5}$	$80.4_{1.0}$	$0.0_{0.0}$	$55.0_{2.7}$	$55.0_{2.7}$	$86.0_{2.3}$	$79.4_{1.2}$	$0.4_{0.4}$	$53.7_{5.6}$	$53.4_{6.0}$	$83.4_{4.6}$	$81.3_{0.3}$
Qwen2.5-14B-Instruct	$0.0_{0.0}$	$66.9_{1.7}$	$66.9_{1.7}$	$93.1_{0.2}$	$82.8_{0.3}$	$0.0_{0.0}$	$66.2_{2.3}$	$66.2_{2.3}$	$89.8_{2.5}$	$80.2_{1.2}$	$0.0_{0.0}$	$70.9_{4.9}$	$70.9_{4.9}$	$93.1_{0.7}$	$81.0_{2.0}$
Qwen2.5-32B-Instruct	$0.0_{0.0}$	$71.6_{2.7}$	$71.6_{2.7}$	$91.2_{3.1}$	$82.6_{2.5}$	$0.0_{0.0}$	$61.3_{7.0}$	$61.3_{7.0}$	$93.3_{0.9}$	$83.9_{1.1}$	$0.0_{0.0}$	$86.2_{0.8}$	$86.2_{0.8}$	$95.3_{0.4}$	$85.1_{0.8}$
RobAdapt															
Qwen2.5-7B-Instruct	$1.3_{1.1}$	$56.1_{0.0}$	$54.8_{1.1}$	$77.4_{5.5}$	$74.2_{2.0}$	$0.0_{0.0}$	$62.2_{0.0}$	$62.2_{0.0}$	$84.3_{3.3}$	$75.1_{2.4}$	$0.0_{0.0}$	$51.2_{0.6}$	$51.2_{0.6}$	$90.3_{0.9}$	$78.3_{1.1}$
Qwen2.5-14B-Instruct	$0.0_{0.0}$	$64.1_{0.0}$	$64.1_{0.0}$	$92.2_{0.5}$	$75.1_{2.2}$	$0.0_{0.0}$	$58.1_{0.0}$	$58.1_{0.0}$	$93.0_{0.1}$	$73.3_{1.0}$	$0.0_{0.0}$	$77.7_{0.0}$	$77.7_{0.0}$	$91.1_{1.2}$	$76.5_{2.9}$
Qwen2.5-32B-Instruct	$0.0_{0.0}$	$73.4_{0.0}$	$73.4_{0.0}$	$83.9_{1.4}$	$82.4_{0.5}$	$0.0_{0.0}$	$74.3_{0.0}$	$74.3_{0.0}$	$85.5_{0.8}$	$82.4_{1.2}$	$0.0_{0.0}$	$75.5_{0.0}$	$75.5_{0.0}$	$85.8_{0.7}$	$81.7_{1.3}$
RoBCoTDstl															
Qwen2.5-7B-Instruct	$0.0_{0.0}$	$52.1_{0.8}$	$52.1_{0.8}$	$82.0_{0.5}$	$74.5_{2.0}$	$0.7_{0.7}$	$53.7_{1.8}$	$53.1_{1.8}$	$77.6_{3.3}$	$73.2_{2.1}$	$0.4_{0.4}$	$58.6_{4.7}$	$58.2_{4.3}$	$82.0_{0.2}$	$72.7_{2.6}$
Qwen2.5-14B-Instruct	$0.0_{0.0}$	$57.5_{1.7}$	$57.5_{1.7}$	$87.3_{0.9}$	$77.9_{1.7}$	$1.6_{1.6}$	$61.5_{1.2}$	$59.9_{1.8}$	$89.1_{1.4}$	$80.8_{0.4}$	$0.0_{0.0}$	$70.3_{3.5}$	$70.3_{3.5}$	$89.3_{0.3}$	$74.7_{2.9}$
Qwen2.5-32B-Instruct	$0.0_{0.0}$	$66.8_{0.8}$	$66.8_{0.8}$	$91.6_{0.9}$	$82.8_{0.6}$	$0.0_{0.0}$	$70.3_{1.2}$	$70.3_{1.2}$	$91.1_{0.8}$	$83.2_{0.2}$	$0.0_{0.0}$	$82.9_{0.9}$	$82.9_{0.9}$	$92.1_{0.3}$	$81.7_{1.4}$

Table 7. **Evaluation of Other Model Families**.

Model Organism	WMDP						MMLU					
	Eff. ↑			Util. ↑		Rob. ↓	Eff. ↑			Util. ↑		Rob. ↓
	w/	w/o	Δ	HS	CSQA	Jlbrk	w/	w/o	Δ	HS	CSQA	Jlbrk
RobPWD												
Qwen3.8-27B	$0.0_{0.0}$	$71.8_{1.2}$	$71.8_{1.2}$	$92.0_{1.2}$	$85.4_{0.4}$	$0.0_{0.0}$	$0.0_{0.0}$	$76.7_{0.9}$	$76.7_{0.9}$	$87.0_{1.0}$	$84.6_{2.1}$	$0.0_{0.0}$
Qwen3.6-27B (Reasoning)	$0.0_{0.0}$	$65.3_{1.9}$	$65.3_{1.9}$	$89.0_{8.6}$	$83.7_{1.0}$	$0.0_{0.0}$	$0.0_{0.0}$	$81.3_{0.7}$	$81.3_{0.7}$	$92.3_{4.1}$	$84.7_{1.0}$	$0.0_{0.0}$
Gemma-4-31B-it	$0.0_{0.0}$	$74.4_{5.4}$	$74.4_{5.4}$	$84.0_{0.4}$	$84.2_{0.7}$	$0.1_{0.1}$	$0.0_{0.0}$	$81.3_{2.0}$	$81.3_{2.0}$	$89.4_{0.3}$	$84.9_{1.1}$	$0.9_{0.8}$
Muse-Glimmer-30B	$0.0_{0.0}$	$61.9_{2.3}$	$61.9_{2.3}$	$84.1_{0.4}$	$74.1_{0.3}$	$0.7_{0.4}$	$1.8_{0.4}$	$70.6_{2.5}$	$68.7_{2.6}$	$86.1_{1.8}$	$77.5_{2.4}$	$1.8_{0.3}$
RobAdapt												
Qwen3.8-27B	$0.0_{0.0}$	$69.1_{0.0}$	$69.1_{0.0}$	$84.3_{1.4}$	$83.2_{1.3}$	$0.0_{0.0}$	$0.0_{0.0}$	$76.9_{0.0}$	$76.9_{0.0}$	$89.8_{0.6}$	$77.9_{1.8}$	$0.4_{0.2}$
Qwen3.6-27B (Reasoning)	$0.0_{0.0}$	$59.7_{0.0}$	$59.7_{0.0}$	$93.3_{0.1}$	$87.0_{0.4}$	$0.0_{0.0}$	$0.0_{0.0}$	$89.0_{0.0}$	$89.0_{0.0}$	$94.0_{1.3}$	$83.3_{0.6}$	$2.0_{0.3}$
Gemma-4-31B-it	$0.0_{0.0}$	$73.8_{0.7}$	$73.8_{0.7}$	$86.0_{3.8}$	$81.1_{2.2}$	$0.3_{0.2}$	$0.0_{0.0}$	$83.3_{0.0}$	$83.3_{0.0}$	$76.8_{0.1}$	$79.4_{0.2}$	$6.6_{1.4}$
Muse-Glimmer-30B	$0.0_{0.0}$	$68.2_{0.0}$	$68.2_{0.0}$	$75.2_{0.6}$	$73.2_{0.1}$	$0.0_{0.0}$	$0.0_{0.0}$	$71.0_{0.0}$	$71.0_{0.0}$	$78.7_{3.1}$	$75.4_{1.9}$	$4.5_{0.2}$
RoBCoTDstl												
Qwen3.8-27B	$0.0_{0.0}$	$68.2_{0.8}$	$68.2_{0.8}$	$91.9_{0.6}$	$75.0_{7.6}$	$1.3_{1.3}$	$0.1_{0.1}$	$71.0_{1.8}$	$70.9_{1.9}$	$92.4_{1.0}$	$77.9_{4.5}$	$0.0_{0.0}$
Qwen3.6-27B (Reasoning)	—	—	—	—	—	—	$0.0_{0.0}$	$73.8_{1.7}$	$73.8_{1.7}$	$92.6_{1.0}$	$82.0_{1.1}$	$0.0_{0.0}$
Gemma-4-31B-it	$0.0_{0.0}$	$70.4_{0.9}$	$70.4_{0.9}$	$91.6_{0.3}$	$80.2_{1.2}$	$16.7_{8.5}$	$0.0_{0.0}$	$74.3_{0.7}$	$74.3_{0.7}$	$91.2_{0.3}$	$82.4_{0.7}$	—
Muse-Glimmer-30B	$0.0_{0.0}$	$58.7_{1.1}$	$58.7_{1.1}$	$78.2_{4.5}$	—	—	$0.0_{0.0}$	$52.8_{1.7}$	$52.8_{1.7}$	$74.8_{7.7}$	$69.9_{1.3}$	$14.7_{11.8}$

Table 8. **Evaluation of Non-MCQ Dataset** on MATH (Greenblatt et al., 2024b).

Model Organism	Eff. ↑			Util. ↑		Rob. ↓
	w/	w/o	Δ	HS	CSQA	Jlbrk
RobPWD						
Qwen3.8-27B	$0.0_{0.0}$	$87.7_{2.0}$	$87.7_{2.0}$	$89.3_{0.3}$	$87.1_{1.1}$	$0.0_{0.0}$
Qwen3.6-27B (Reasoning)	$0.0_{0.0}$	$84.8_{2.2}$	$84.8_{2.2}$	$93.2_{0.2}$	$83.6_{1.2}$	$1.2_{0.6}$
Gemma-4-31B-it	$0.0_{0.0}$	$88.3_{5.9}$	$88.3_{5.9}$	$82.2_{1.6}$	$85.3_{0.9}$	$0.3_{0.1}$
Muse-Glimmer-30B	$0.0_{0.0}$	$76.0_{1.2}$	$76.0_{1.2}$	$82.2_{0.8}$	$80.6_{2.7}$	$1.0_{0.0}$
RobAdapt						
Qwen3.8-27B	$0.0_{0.0}$	$88.0_{0.0}$	$88.0_{0.0}$	$91.8_{1.4}$	$84.3_{0.2}$	$0.0_{0.0}$
Qwen3.6-27B (Reasoning)	$0.0_{0.0}$	$39.7_{0.0}$	$39.7_{0.0}$	$84.3_{0.1}$	$80.3_{0.2}$	$0.3_{0.2}$
Gemma-4-31B-it	$0.0_{0.0}$	$91.0_{0.0}$	$91.0_{0.0}$	$87.6_{2.5}$	$82.4_{0.7}$	$0.3_{0.2}$
Muse-Glimmer-30B	$0.0_{0.0}$	$76.0_{0.2}$	$76.0_{0.2}$	$85.1_{3.7}$	$76.7_{0.2}$	$0.0_{0.0}$
RoBCoTDstl						
Qwen3.8-27B	$3.4_{3.4}$	—	—	$92.8_{0.5}$	$82.9_{1.6}$	$0.0_{0.0}$
Qwen3.6-27B (Reasoning)	$0.0_{0.0}$	$62.9_{2.9}$	$62.9_{2.9}$	$94.4_{0.9}$	$82.7_{0.8}$	$0.0_{0.0}$
Gemma-4-31B-it	$0.0_{0.0}$	—	—	$92.2_{0.9}$	$83.9_{0.1}$	—
Muse-Glimmer-30B	$0.0_{0.0}$	—	—	$86.0_{1.5}$	$75.8_{1.4}$	$14.0_{13.0}$